

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I N. Meghana (192411129) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N. Meghana

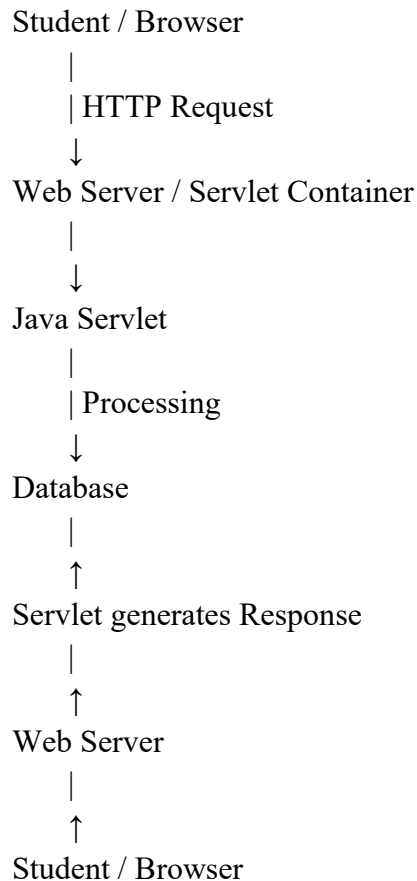
RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. A university wants to build an online system for registering students where they can enter their information via a web form and this input can be processed by using a Java servlet.

Architecture of Servlet-Based Web Application

Servlet based web applications have a client/server architecture.



Components

Client/Browser

- Student provides his/her registration information through HTML forms.
- The browser transmits the HTML form data to the server.

Web Server/Servlet Container

- Accepts the HTTP request.
- Apache Tomcat is one example.
- Manages the servlet life cycle.

Servlet

- Handles the request.
- Parses the form data through `request.getParameter()` method.
- Does the necessary processing.

Database

- Keeps the data of the student if necessary.

HTTP Response

- Servlet returns the result through HttpServletResponse object.

HTML Form

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
</head>
<body>

<h2>Student Registration</h2>

<form action="student" method="post">
  Name:
  <input type="text" name="name"><br><br>

  Register Number:
  <input type="text" name="regno"><br><br>

  Email:
  <input type="email" name="email"><br><br>

  <input type="submit" value="Register">
</form>

</body>
</html>
```

Servlet Using GET and POST

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```

String name = request.getParameter("name");
String regno = request.getParameter("regno");
String email = request.getParameter("email");

out.println("<h2>Student Details - GET</h2>");
out.println("Name: " + name + "<br>");
out.println("Register Number: " + regno + "<br>");
out.println("Email: " + email);
}

public void doPost(HttpServletRequest request,
                    HttpServletResponse response)
                    throws ServletException, IOException {

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    String name = request.getParameter("name");
    String regno = request.getParameter("regno");
    String email = request.getParameter("email");

    out.println("<h2>Registration Successful</h2>");
    out.println("Name: " + name + "<br>");
    out.println("Register Number: " + regno + "<br>");
    out.println("Email: " + email);
}
}

```

- **web.xml**

```

<web-app>

    <servlet>
        <servlet-name>StudentServlet</servlet-name>
        <servlet-class>StudentServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>StudentServlet</servlet-name>
        <url-pattern>/student</url-pattern>
    </servlet-mapping>

</web-app>

```

GET VS POST

GET	POST
Data is placed in URL	Data is in request body
Data is apparent in URL	Data is usually hidden in URL
Useful for fetching data	Useful for sending data
Limited size of URL	Can transfer larger amount of data
Can be saved in bookmarks	Usually, cannot be saved with the form

Which of the two is more appropriate for transmission of sensitive data?

POST is better for sensitive data since the form data will be placed in the HTTP request body. However, posting does not guarantee encryption. Sensitive data must be transferred with the use of HTTPS.

Lifecycle of a Servlet

1. init()

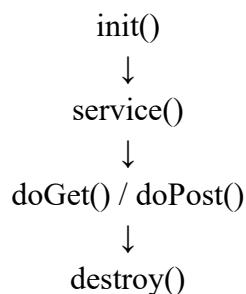
- The initialization phase occurs when the servlet is loaded.
- Initialization requires the resources to be initialized.

2. service()

- The processing phase occurs when the servlet is accessed.
- During the processing phase requests are processed and doGet() or doPost() methods are invoked.

3. destroy()

- The clean-up phase occurs when the servlet is destroyed.
- During this phase the servlet is removed from memory.



Q2. Session Management Using HttpSession and Cookies

When a user logs in to the website, the website should remember the user as he/she continues to browse through different pages.

What is the need for session management?

One of the significant disadvantages of http is that it does not have this capability by default.

For instance:

Login --> Home --> Profile --> Cart

The website must keep track of the original user in each of these pages.

This is what is referred to as session tracking.

The concept of HttpSession.

HttpSession provides the platform for storing the user's information using their visits.

After Login

```
HttpSession session = request.getSession();
```

```
session.setAttribute("username", username);
```

The username is now stored in the session.

Getting the Username

On another page:

```
HttpSession session = request.getSession(false);
```

```
String username =
```

```
    (String) session.getAttribute("username");
```

```
out.println("Welcome " + username);
```

The user can now move to different pages without logging in again.

Logout

```
HttpSession session = request.getSession(false);
```

```
if (session != null) {  
    session.invalidate();  
}
```

This removes the user's session.

Using Cookies

A cookie is a small piece of information stored in the user's browser.

Creating a Cookie

```
Cookie c = new Cookie("username", username);
```

```
response.addCookie(c);
```

Reading a Cookie

```
Cookie[] cookies = request.getCookies();
```

```
if (cookies != null) {  
  
    for (Cookie c : cookies) {  
  
        if (c.getName().equals("username")) {  
            String name = c.getValue();  
            out.println("Welcome " + name);  
        }  
    }  
}
```

```
}  
}
```

HttpSession	Cookies
Data is mainly stored on server	Data is stored in browser
Good for login sessions	Good for small information
More suitable for user sessions	Useful for preferences
Session can be removed using invalidate()	Cookie has an expiry time

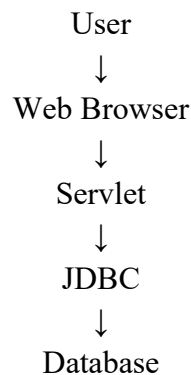
Conclusion

For a login system, **HttpSession is better for maintaining the user's login session**. Cookies can also be used for session tracking or storing simple preferences.

Q3. E-Commerce Application Using JDBC

The e-commerce website needs to store product details in a database and display them when required.

Basic Architecture



The user sends a request from the browser. The Servlet receives it and uses JDBC to communicate with the database.

Product Table

```
CREATE TABLE product (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    price DOUBLE  
);
```

Insert Product

```
String sql =  
    "INSERT INTO product VALUES (?, ?, ?)";
```

```
PreparedStatement ps =  
    con.prepareStatement(sql);
```

```
ps.setInt(1, 1);
```



```
ps.setString(2, "Laptop");
ps.setDouble(3, 50000);
```

```
ps.executeUpdate();
```

This adds a product to the database.

Retrieve Products

```
String sql = "SELECT * FROM product";
```

```
PreparedStatement ps =
    con.prepareStatement(sql);
```

```
ResultSet rs = ps.executeQuery();
```

```
while (rs.next()) {
```

```
    int id = rs.getInt("id");
    String name = rs.getString("name");
    double price = rs.getDouble("price");
```

```
        System.out.println(id + " " + name + " " + price);
    }
```

This gets the product details from the database.

JDBC Steps

There are mainly **5 steps**:

1. Load the Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Connect to Database

```
Connection con =
    DriverManager.getConnection(
        url, username, password);
```

3. Create SQL Statement

```
PreparedStatement ps =
    con.prepareStatement(sql);
```

4. Execute Query

For inserting:

```
ps.executeUpdate();
```

For retrieving:

```
ResultSet rs = ps.executeQuery();
```

5. Close Connection

```
con.close();
```

Exception Handling

Database errors can happen, so we use try-catch.

```
try {
```

```
// database operations

}
catch (SQLException e) {

    System.out.println("Database error");
}
PreparedStatement is preferred because it makes it safer to work with values entered by users.
```